K L Deemed to be University

Name: K.Pranav

Roll No: 2520030477

DATA STRUCTURES AND ALGORITHMS - 2

CO1 to CO3 Case Study Question Bank with Solution Keys

QUESTION PAPER STYLE CASE STUDIES

BST/AVL Trees • Segment Trees/Fenwick Trees • Graph Traversals & MST

Course	Data Structures and Algorithms - 2	**Mode**	Project-Based Learning
Coverage	CO1, CO2, CO3	**BTL**	Level 4 - Apply/Analyze
Total Case Studies	3 complete case studies	**Marks Pattern**	5 + 7 + 3 = 15 Marks each

Document includes real-world scenarios, diagrams, sub-questions, and detailed solution keys.

Course Outcome Mapping

CO No	Selected Topic	Case Study Theme	BTL
CO1	Binary Search Trees and AVL Trees	Hospital patient-ID index with AVL rebalancing	4
CO2	Segment Trees	Smart warehouse range-query dashboard with point updates	4
CO3	Minimum Spanning Tree using Kruskal/Prim	Campus fiber network design under cost constraints	4

CASE STUDY 1 - CO1: BST and AVL Tree Indexing

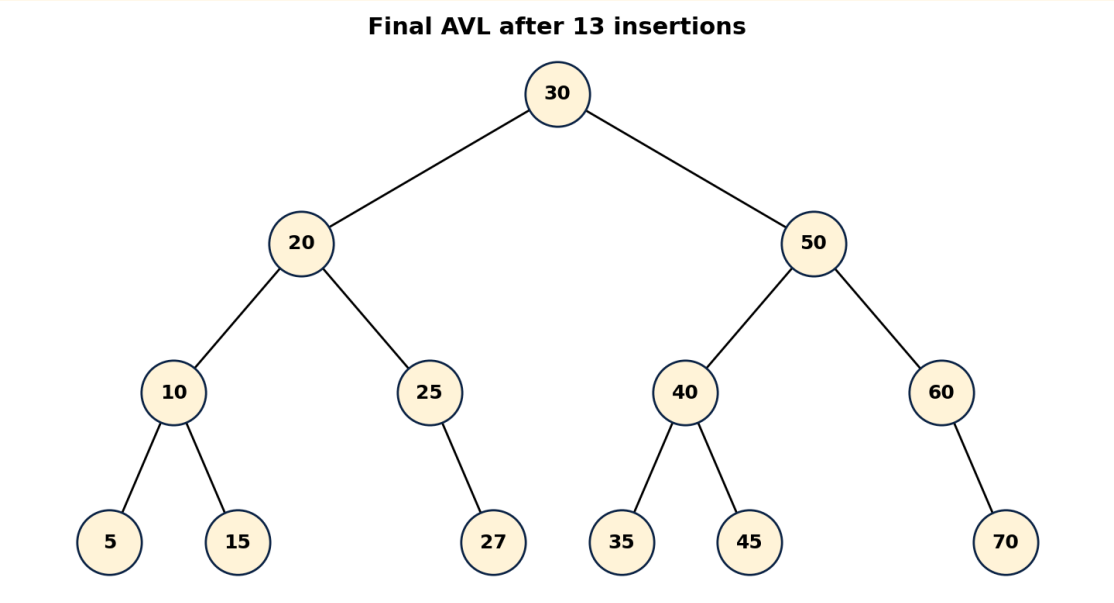
CO	CO1	**Topic**	BST and AVL Trees
Scenario	MediFlow Hospital patient dictionary	**Marks**	15
BTL	4	**Main Skill**	Construct, rebalance, and analyze AVL operations

SCENARIO / CASE STUDY

MediFlow Hospital uses an 8-digit Patient-ID dictionary to answer triage lookups. IDs arrive in the order appointments are registered. During the morning block, most IDs are nearly sorted because older appointments are processed first.

Today, 13 Patient-IDs arrive in this order: 40, 20, 60, 10, 30, 50, 70, 25, 35, 5, 15, 27, 45. The hospital wants point-lookups to remain fast even after continuous insertions.

- Each pointer hop is approximately 200 ns.
- The triage lookup SLA is p99 < 5 ms.
- A plain BST may become skewed when data is monotonic; an AVL tree must maintain balance factor -1, 0, or +1.



Sub	Question	Marks	CO/BTL
(a)	Construct the plain BST from the given insertion sequence. State the final tree height in edges and estimate the worst-case lookup time using the 200 ns per hop model. Explain whether the tree is structurally safe for the SLA.	5	CO1/BTL4
(b)	Build the AVL tree from the same insertion sequence. After insertions that trigger rebalancing, name the rotation type and pivot node. Then complete the insert, rotateLeft, and rotateRight logic in Java-style pseudocode.	7	CO1/BTL4
(c)	After noon deletions of 10, 60, and 40, show the expected effect on balance and state whether the AVL tree still guarantees O(log n) lookup. Identify the deletion likely to cause the largest structural change.	3	CO1/BTL4

Solution Key - Case Study 1

Answer (a) - Plain BST construction and SLA check

Insertion behavior: The first seven values form a balanced root area, but the later values attach as leaf refinements under 30, 10, and 50.

Height: The longest root-to-leaf path is 40 -> 20 -> 30 -> 25 -> 27, so height = 4 edges.

Worst-case lookup: A lookup may traverse 5 nodes = 5 pointer hops approximately. 5 x 200 ns = 1000 ns = 0.001 ms.

SLA result: For this small dataset, the tree is within 5 ms. However, a plain BST has no structural guarantee; if future IDs arrive sorted, the height can become n-1 and the SLA can fail at scale.

Answer (b) - AVL rotations and code

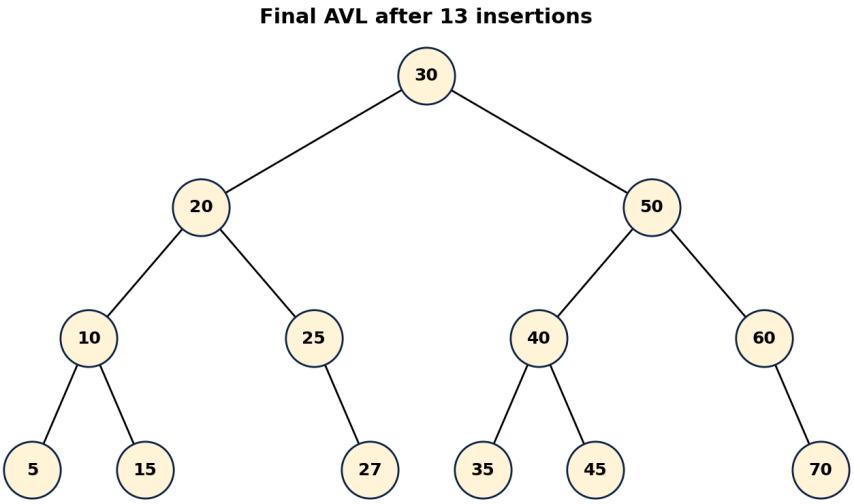
Rotation trace: insert 27: LL/left rotation at pivot 20, new subtree root 30; insert 27: RR/right rotation at pivot 40, new subtree root 30; insert 45: RR/right rotation at pivot 60, new subtree root 50; insert 45: LL/left rotation at pivot 40, new subtree root 50

Final AVL height: The final AVL tree height is 3 edges for 13 nodes, so the lookup path is logarithmic and stable.

Key idea: After every insertion, update height, compute balance factor, and apply one of LL, RR, LR, or RL rotations only at the first unbalanced ancestor.

```
static AVLNode rotateRight(AVLNode y) {
    AVLNode x = y.left;
    AVLNode T2 = x.right;
    x.right = y;
    y.left = T2;
    updateHeight(y);
    updateHeight(x);
    return x;
}

static AVLNode rotateLeft(AVLNode x) {
    AVLNode y = x.right;
    AVLNode T2 = y.left;
    y.left = x;
    x.right = T2;
    updateHeight(x);
    updateHeight(y);
    return y;
}

static AVLNode insert(AVLNode node, int key) {
    if (node == null) return new AVLNode(key);
```

```
    if (key < node.key) node.left = insert(node.left, key);
    else if (key > node.key) node.right = insert(node.right, key);
    else return node; // no duplicates

    updateHeight(node);
    int bf = balance(node);

    if (bf > 1 && key < node.left.key) return rotateRight(node);      // LL
    if (bf < -1 && key > node.right.key) return rotateLeft(node);     // RR
    if (bf > 1 && key > node.left.key) {                              // LR
        node.left = rotateLeft(node.left);
        return rotateRight(node);
    }
    if (bf < -1 && key < node.right.key) {                            // RL
        node.right = rotateRight(node.right);
        return rotateLeft(node);
    }
    return node;
}
```



Answer (c) - Deletion effect

Deletion 10: It removes an internal node with children 5 and 15, so the usual replacement is inorder successor/predecessor. Local balance may change in the left subtree.

Deletion 60: Node 60 has children 50 and 70. Replacing it with successor 70 or predecessor 50 changes the right subtree but remains repairable through AVL height updates.

Deletion 40: Deleting the root usually causes the largest structural change because the root must be replaced, and height/balance must be recomputed from the replacement path upward.

Guarantee: After each delete, AVL rebalancing restores balance factor to -1, 0, or +1, so lookup remains O(log n).

Roll No: 2520030477

CASE STUDY 2 - CO2: Segment Tree Range Queries

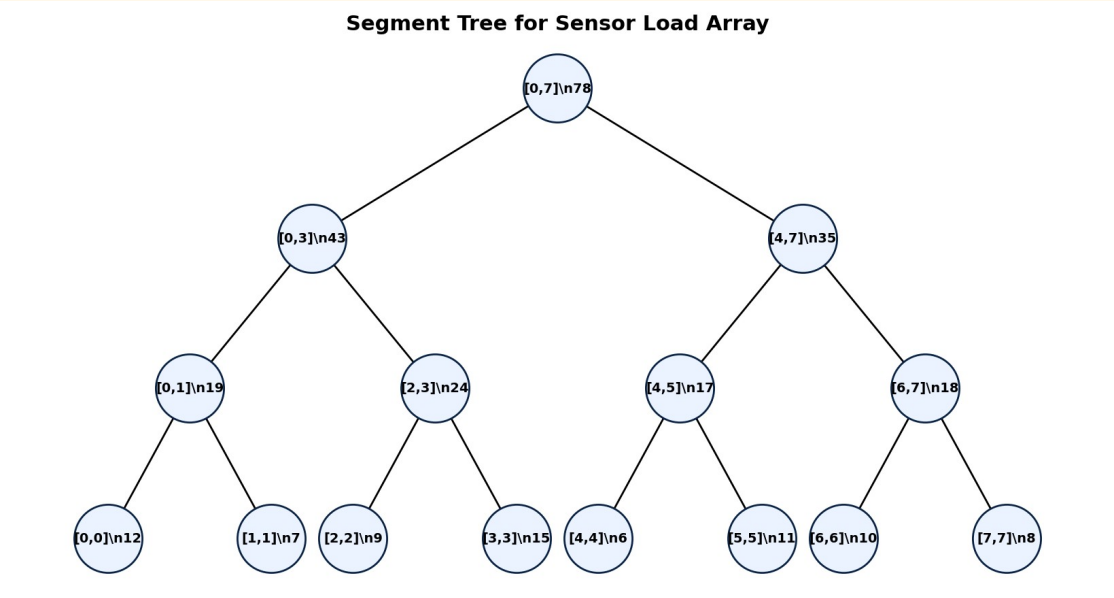
CO	CO2	**Topic**	Segment Trees
Scenario	Smart warehouse load-monitoring dashboard	**Marks**	15
BTL	4	**Main Skill**	Build, query, update, and analyze range structures

SCENARIO / CASE STUDY

A smart warehouse stores load readings from 8 rack sensors. The dashboard must answer range-sum queries quickly and also update a sensor reading whenever a rack load changes.

Initial sensor array by rack index 0 to 7 is: [12, 7, 9, 15, 6, 11, 10, 8]. The system receives range queries Q1 = sum(2, 6), Q2 = sum(0, 3), then a point update index 3 from 15 to 4, followed by Q3 = sum(2, 6).

- A naive range query scans every element in the interval.
- The dashboard prefers O(log n) query and update performance.
- Use a sum Segment Tree with 0-based indices.



Sub	Question	Marks	CO/BTL
(a)	Construct the segment tree for the given array. Write the root sum and the major internal node sums for intervals [0,3], [4,7], [0,1], [2,3], [4,5], and [6,7].	5	CO2/BTL4
(b)	Trace Q1 = sum(2,6), Q2 = sum(0,3), update index 3 from 15 to 4, and Q3 = sum(2,6). Mention the nodes used or recomputed during query/update.	7	CO2/BTL4
(c)	Compare Segment Tree and Fenwick Tree for this dashboard. State which one is more suitable if the system later needs range minimum query instead of only sum.	3	CO2/BTL4

Roll No: 2520030477

Solution Key - Case Study 2

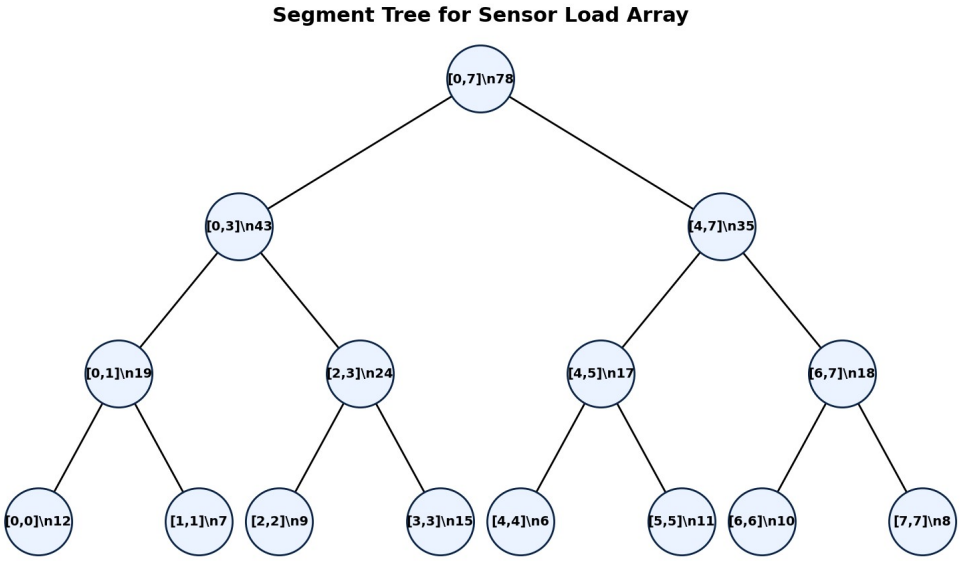
Answer (a) - Segment tree construction

Root: Total sum [0,7] = 12 + 7 + 9 + 15 + 6 + 11 + 10 + 8 = 78.

Internal sums: [0,3] = 43, [4,7] = 35, [0,1] = 19, [2,3] = 24, [4,5] = 17, [6,7] = 18.

Leaves: The leaves are the original array values: 12, 7, 9, 15, 6, 11, 10, 8.

Height: For 8 elements, the complete segment tree has height log2(8) = 3 edges.



Answer (b) - Query and update trace

Q1 sum(2,6): Use [2,3] = 24, [4,5] = 17, and [6,6] = 10. Total = 24 + 17 + 10 = 51.

Q2 sum(0,3): The interval [0,3] is a total-overlap node, so the answer is 43 directly.

Update index 3: Value changes from 15 to 4. Recompute [2,3] from 9 + 4 = 13, recompute [0,3] from 19 + 13 = 32, and recompute root [0,7] from 32 + 35 = 67.

Q3 sum(2,6): After update, use [2,3] = 13, [4,5] = 17, and [6,6] = 10. Total = 40.

Complexity: Each query/update visits only O(log n) relevant tree levels, not the full array.

```
int query(node, L, R, ql, qr) {
   if (qr < L || R < ql) return 0;        // no overlap
   if (ql <= L && R <= qr) return tree[node]; // total overlap
   int mid = (L + R) / 2;
   return query(2*node, L, mid, ql, qr)
       + query(2*node+1, mid+1, R, ql, qr);
}

void update(node, L, R, idx, newVal) {
   if (L == R) { tree[node] = newVal; return; }
   int mid = (L + R) / 2;
   if (idx <= mid) update(2*node, L, mid, idx, newVal);
   else update(2*node+1, mid+1, R, idx, newVal);
   tree[node] = tree[2*node] + tree[2*node+1];
}
```

Answer (c) - Segment Tree vs Fenwick Tree

Fenwick Tree: Best for prefix sums, range sums, and point updates with low memory and simple implementation.

Segment Tree: More flexible; supports sums, minimums, maximums, gcd, lazy propagation, and custom merge functions.

Choice: For only sums, Fenwick Tree is lighter. If the dashboard later needs range minimum query, Segment Tree is more suitable because the same tree idea can merge minimum values naturally.

Roll No: 2520030477

CASE STUDY 3 - CO3: Minimum Spanning Tree for Network Design

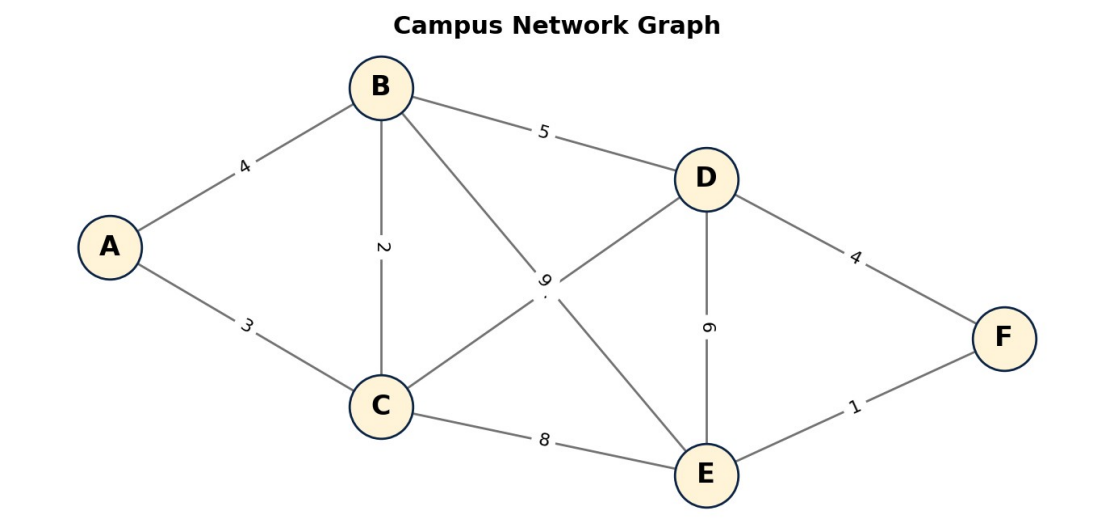
CO	CO3	**Topic**	MST using Kruskal and Prim
Scenario	Campus fiber network design	**Marks**	15
BTL	4	**Main Skill**	Apply MST algorithms and justify edge selection

SCENARIO / CASE STUDY

A university wants to connect six buildings with fiber cables. The network team must connect every building with minimum total cable cost while avoiding cycles. The buildings are A, B, C, D, E, and F.

Available cable routes and costs are: AB=4, AC=3, BC=2, BD=5, CD=7, CE=8, DE=6, DF=4, EF=1, BE=9.

- The final network must connect all buildings.
- No redundant cycles are allowed in the minimum design.
- Use Kruskal or Prim to obtain a Minimum Spanning Tree.



Sub	Question	Marks	CO/BTL
(a)	Draw/represent the weighted undirected graph and sort all edges in non-decreasing order of cost. Explain why a spanning tree for 6 buildings must contain exactly 5 edges.	5	CO3/BTL4
(b)	Apply Kruskal's algorithm step by step. At each step, mention whether the edge is accepted or rejected and give the final MST cost.	7	CO3/BTL4
(c)	If route EF becomes unavailable due to construction, state the new MST adjustment and explain how the total cost changes.	3	CO3/BTL4

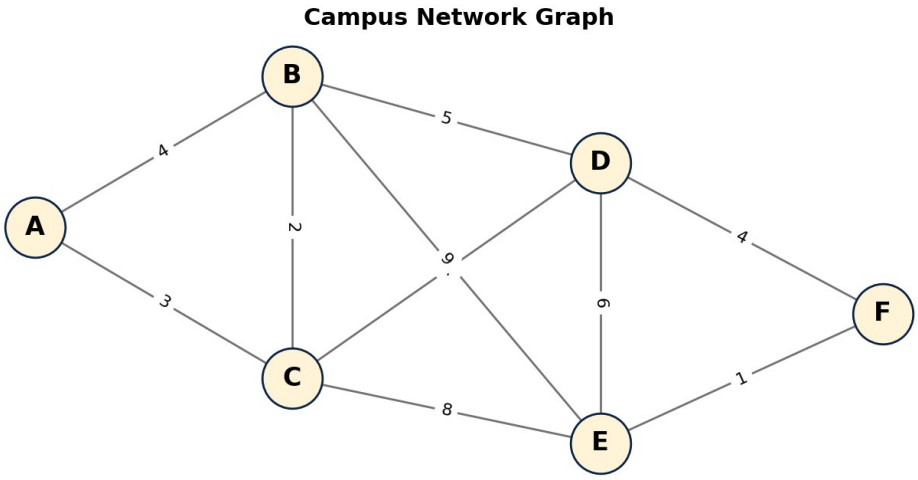
Solution Key - Case Study 3

Answer (a) - Graph representation and edge order

Sorted edges: EF(1), BC(2), AC(3), AB(4), DF(4), BD(5), DE(6), CD(7), CE(8), BE(9).

Spanning tree size: For n vertices, a tree has n - 1 edges. Here n = 6, so the MST must have 5 edges.

Reason: Fewer than 5 edges cannot connect all 6 buildings; more than 5 edges creates at least one cycle.



Answer (b) - Kruskal MST trace

EF(1): Accept. It connects E and F.

BC(2): Accept. It connects B and C.

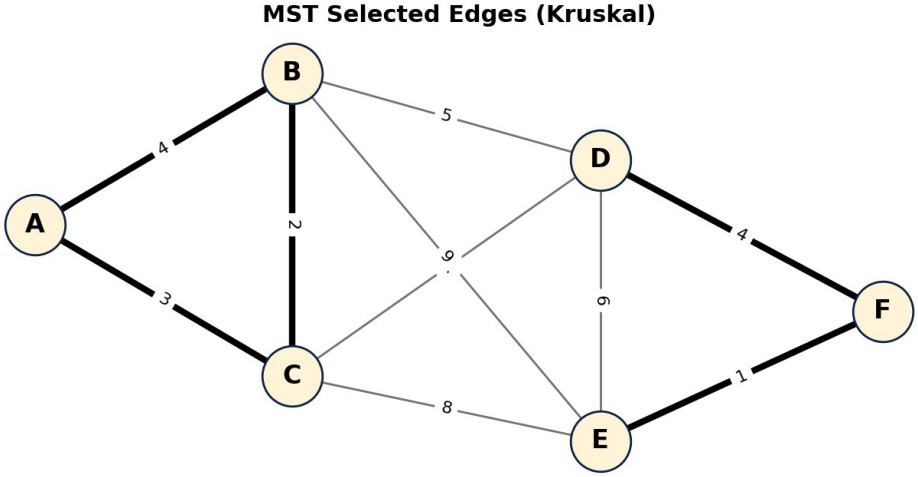
AC(3): Accept. It connects A to the B-C component.

AB(4): Reject. A, B, and C are already connected, so AB forms a cycle.

DF(4): Accept. It connects D to the E-F component.

BD(5): Accept. It connects the A-B-C component to the D-E-F component. Now all 6 buildings are connected with 5 edges.

Final MST: {EF, BC, AC, DF, BD}. Total cost = 1 + 2 + 3 + 4 + 5 = 15.



Answer (c) - Route failure adjustment

Unavailable edge: If EF(1) is removed, the cheapest way to connect E is usually DE(6) because CE(8) and BE(9) are costlier.

New MST: A valid new MST is {BC(2), AC(3), DF(4), BD(5), DE(6)}.

Cost change: New total = 2 + 3 + 4 + 5 + 6 = 20. The cost increases by 5 compared with the original MST cost 15.

Reason: The removed edge EF was the cheapest bridge for E-F. Replacing it with DE forces a more expensive connection while

maintaining connectivity and no cycles.

Final Submission Checklist

QUALITY CHECK

This document contains one case study from each of CO1, CO2, and CO3. Each case study follows a 15-mark pattern with a scenario, diagram, sub-questions, and complete solution key.

- CO mapping included.
- BTL 4 included.
- Diagrams included for tree, segment tree, and graph/MST.
- Solutions include construction, algorithm trace, code/pseudocode, and complexity/justification.